Concurrent Programming – Classical Problems Exam with Answers

EXAM QUESTIONS
==========

Instructions:
Fill in each blank (___) with the correct keyword, operation, expression, or identifier.
Assume a C-like pseudocode unless stated otherwise.

COVERED TOPICS
- Semaphores
- Mutex locks
- Peterson's solution
- Monitors
- Classical problems: Producer–Consumer, Readers–Writers, Dining Philosophers

SECTION 1: SEMAPHORES & PRODUCER–CONSUMER
---

1. Basic semaphore operations

A semaphore `S` is initialized to 1 for mutual exclusion. Fill in the standard names
of the two atomic operations used to decrement and increment `S`:

a) Decrement (may block): _____(S);
b) Increment (may wake a process): _____(S);

2. Bounded buffer with counting semaphore

We implement a producer using the counting semaphore `empty`, the counting semaphore
`full`, and a binary semaphore `mutex` to protect the buffer.

Producer pseudocode:

```
produce(item);
_____(empty);      // (1) wait on "empty" slots
_____(mutex);      // (2) enter critical section
buffer[in] = item;
in = (in + 1) % N;
_____(mutex);      // (3) leave critical section
_____(full);       // (4) signal that there is a new "full" slot
```

Fill in blanks (1)–(4) with either `wait` or `signal` and the correct semaphore name,
e.g., `wait(empty)`.

(1) _____
(2) _____
(3) _____
(4) _____

3. Binary vs counting semaphore

A semaphore that can take only the values 0 or 1 (and is typically used for mutual exclusion) is called a _____ semaphore. A semaphore that may hold any nonnegative integer value is called a _____ semaphore.

SECTION 2: MUTEX LOCKS & READERS–WRITERS
---

4. Pthread-style mutex usage

Complete the missing function names to implement correct critical-section protection using a POSIX-style mutex `pthread_mutex_t lock`:

```
// shared data
int shared_counter = 0;
pthread_mutex_t lock;

void *worker(void *arg) {
    _____(&lock);      // (1)
    shared_counter++;
    _____(&lock);      // (2)
    return NULL;
}
```

Fill in (1) and (2) with the correct pthread mutex functions.

5. Readers–writers, first readers-preference solution (simplified)

We use:
- `mutex` (binary semaphore) to protect `readcount`
- `wrt` (binary semaphore) to provide writers exclusive access

Shared variables:

```
int readcount = 0;
semaphore mutex    = 1;
semaphore wrt      = 1;
```

Reader pseudocode:

```
wait(_________);                    // (1)
readcount = readcount + 1;
if (readcount == 1)
    wait(_________);                // (2)
signal(_________);                   // (3)

// --- critical section (reading) ---
read_data();

wait(_________);                    // (4)
```

```
    readcount = readcount - 1;
    if (readcount == 0)
        signal(_________);              // (5)
    signal(_________);                  // (6)
```

Fill in blanks (1)–(6) with either `mutex` or `wrt` to implement the readers-preference solution correctly.

(1) _________
(2) _________
(3) _________
(4) _________
(5) _________
(6) _________

SECTION 3: PETERSON'S SOLUTION

Assume two processes P0 and P1 with the shared variables:

```
    boolean flag[2];
    int turn;
```

6. Initialization

To use Peterson's solution for mutual exclusion between P0 and P1, the shared array `flag` should initially be set to _____ and the variable `turn` may be initialized to either ____ or ____ (any one process index).

7. Entry section for process Pi (i is 0 or 1)

Complete the entry section of Peterson's two-process solution for process `i`:

```
    // i is 0 or 1; j = 1 - i
    flag[i] = _________;          // (1)
    turn    = _________;          // (2)
    while ( flag[_________] && turn == _________ ) {
        ; // busy wait
    }
```

Fill in the blanks (1)–(4) with appropriate values or expressions involving `true`, `false`, `i`, and `j`.

8. Mutual exclusion property

In Peterson's solution for two processes, mutual exclusion is guaranteed because at most _____ process(es) can be in the critical section at a time, and the variable _____ is used to break ties when both processes want to enter at once.

SECTION 4: MONITORS & BOUNDED BUFFER

Assume a monitor for a bounded buffer with the following structure (in Java-like pseudocode):

```
monitor BoundedBuffer {
    final int N = 10;
    int count = 0, in = 0, out = 0;
    Item[] buffer = new Item[N];
    condition notFull, notEmpty;

    public void deposit(Item x) {
        while (count == N) {
            notFull._____();    // (1)
        }
        buffer[in] = x;
        in = (in + 1) % N;
        count++;
        notEmpty._____();       // (2)
    }

    public Item remove() {
        while (count == 0) {
            notEmpty._____();   // (3)
        }
        Item x = buffer[out];
        out = (out + 1) % N;
        count--;
        notFull._____();        // (4)
        return x;
    }
}
```

9. Condition variable operations

Fill in blanks (1)–(4) with either `wait()` or `signal()` so that the monitor behaves correctly under the producer–consumer pattern.

10. Monitor mutual exclusion

In a monitor, only _____ thread (or process) can be executing inside any of the monitor's procedures at a given time. This property is known as _____ mutual exclusion provided by the monitor itself.

SECTION 5: DINING PHILOSOPHERS

We have five philosophers (0–4) sitting around a circular table. Each philosopher `i` has chopsticks `i` (left) and `(i + 1) % 5` (right).

11. Naïve semaphore solution (possible deadlock)

Assume each chopstick is represented by a binary semaphore `chopstick[5]`, all initialized to 1. A naïve solution for philosopher `i` might be:

```
do {
    think();
    wait(chopstick[i]);                  // (1)
    wait(chopstick[(i + 1) % 5]);        // (2)
    eat();
    signal(chopstick[(i + 1) % 5]);      // (3)
    signal(chopstick[i]);                // (4)
} while (true);
```

a) In this solution, a deadlock can occur if all philosophers execute the `wait` on their _____ chopstick first at the same time.

b) The main four necessary conditions for deadlock described in operating systems textbooks include: mutual exclusion, hold-and-wait, no preemption, and _____.

12. Asymmetric solution (avoiding circular wait)

One classical way to avoid deadlock is to break circular wait by making one philosopher pick up chopsticks in the opposite order. For example:

```
if (i == 0) {
    wait(chopstick[(i + 1) % 5]);        // (1)
    wait(chopstick[i]);                  // (2)
} else {
    wait(chopstick[i]);                  // (3)
    wait(chopstick[(i + 1) % 5]);        // (4)
}
eat();
signal(chopstick[(i + 1) % 5]);
signal(chopstick[i]);
```

In this code, philosopher 0 picks up the _____ chopstick first, while all other philosophers pick up their _____ chopstick first. This breaks the _____ wait condition for deadlock.

13. Dining philosophers using a monitor (high level)

Assume the following Java-like monitor:

```
monitor DiningPhilosophers {
    enum State { THINKING, HUNGRY, EATING };
    State[] state = new State[5];
    condition[] self = new condition[5];

    void pickup(int i) {
```

```
        state[i] = HUNGRY;
        test(i);
        while (state[i] != EATING) {
            self[i]._________();        // (1)
        }
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);            // left neighbor
        test((i + 1) % 5);            // right neighbor
    }

    void test(int i) {
        if (state[i] == HUNGRY &&
            state[(i + 4) % 5] != EATING &&
            state[(i + 1) % 5] != EATING) {
            state[i] = EATING;
            self[i]._________();        // (2)
        }
    }
}
```

Fill in (1) and (2) with either `wait()` or `signal()`.

14. Starvation vs deadlock

Circle the correct term in each blank:

a) A situation where some philosopher may **never** get to eat while others continue to eat
(even though the system as a whole is still moving) is called _____ (starvation / deadlock).

b) A situation where **no** philosopher can make progress (everyone is stuck waiting forever)
is called _____ (starvation / deadlock).

ANSWER KEY AND EXPLANATIONS
==========

SECTION 1: SEMAPHORES & PRODUCER–CONSUMER
---

1. Basic semaphore operations

a) Decrement (may block): wait(S);
b) Increment (may wake a process): signal(S);

Explanation:

The classic Dijkstra operations for semaphores are `wait` (also called `P` or `down`) and `signal` (also called `V` or `up`). `wait` decrements the semaphore and may block if the value would become negative; `signal` increments it and may wake a waiting process.

2. Bounded buffer with counting semaphore

(1) wait(empty)
(2) wait(mutex)
(3) signal(mutex)
(4) signal(full)

Explanation:
The producer must first ensure there is an empty slot (`wait(empty)`), then take the mutex (`wait(mutex)`) before entering the critical section. After inserting the item and updating indices, it releases the mutex (`signal(mutex)`) and notifies that there is one additional full slot (`signal(full)`).

3. Binary vs counting semaphore

Answer:
- A semaphore that can be only 0 or 1 is a binary semaphore.
- A semaphore that may hold any nonnegative integer is a counting semaphore.

Explanation:
Binary semaphores are typically used for mutual exclusion, acting similarly to a mutex. Counting semaphores track a resource count, such as the number of available buffer slots.

SECTION 2: MUTEX LOCKS & READERS–WRITERS
---

4. Pthread-style mutex usage

(1) pthread_mutex_lock(&lock);
(2) pthread_mutex_unlock(&lock);

Explanation:
For POSIX threads, the basic mutex operations are `pthread_mutex_lock` and `pthread_mutex_unlock`. Locking is done before entering the critical section, and unlocking is done after completing the critical section.

5. Readers–writers, first readers-preference solution (simplified)

Correct filling:

(1) mutex
(2) wrt
(3) mutex
(4) mutex

(5) wrt
(6) mutex

So the completed reader pseudocode:

```
wait(mutex);
readcount = readcount + 1;
if (readcount == 1)
    wait(wrt);
signal(mutex);

// critical section (reading)
read_data();

wait(mutex);
readcount = readcount - 1;
if (readcount == 0)
    signal(wrt);
signal(mutex);
```

Explanation:
`mutex` protects `readcount` so that increments/decrements are atomic. The first reader to enter (when `readcount` goes from 0 to 1) waits on `wrt`, blocking writers. The last reader to leave (when `readcount` goes from 1 to 0) signals `wrt`, allowing a waiting writer to proceed.

SECTION 3: PETERSON'S SOLUTION
---

6. Initialization

Answer:
- `flag` should initially be set to false for both entries.
- `turn` may be initialized to either 0 or 1.

Explanation:
Each `flag[i]` indicates whether process `i` wants to enter the critical section. At start-up, neither process is interested, so both are `false`. The initial value of `turn` can be either process index; it only matters when both processes try to enter simultaneously.

7. Entry section for process Pi (i is 0 or 1)

Correct fill-in:

```
flag[i] = true;          // (1)
turn    = j;             // (2)
while ( flag[j] && turn == j ) {
    ; // busy wait
}
```

So the blanks are:

(1) true
(2) j
(3) j
(4) j

Explanation:
In Peterson's solution, process `i` sets its flag `flag[i] = true` to show interest in the critical section, then yields priority to the other process by setting `turn = j` (where `j = 1 - i`). The `while` loop waits as long as the other process is interested (`flag[j]`) and it is that process's turn (`turn == j`). This ensures that if both processes want to enter at the same time, only one proceeds.

8. Mutual exclusion property

Answer:
- At most 1 process can be in the critical section at a time.
- The variable `turn` is used to break ties.

Explanation:
Mutual exclusion means that only one process is in the critical section at any time. In Peterson's algorithm, if both processes are interested (`flag[0]` and `flag[1]` both true), the `turn` variable decides who proceeds, ensuring no simultaneous entry.

SECTION 4: MONITORS & BOUNDED BUFFER
---

9. Condition variable operations

Correct fill-in:

(1) wait()
(2) signal()
(3) wait()
(4) signal()

So the methods are:

```
    notFull.wait();    // (1)
    notEmpty.signal(); // (2)
    notEmpty.wait();   // (3)
    notFull.signal();  // (4)
```

Explanation:
- In `deposit`, when the buffer is full (`count == N`), the producer must wait for space (`notFull.wait()`).
- After adding an item, the producer signals that the buffer is no longer empty (`notEmpty.signal()`).
- In `remove`, when the buffer is empty (`count == 0`), the consumer must wait for items (`notEmpty.wait()`).

- After removing an item, the consumer signals that the buffer is no longer full (`notFull.signal()`).

10. Monitor mutual exclusion

Answer:

- Only one thread (or process) can be executing inside the monitor at a time.
- This property is known as implicit (or automatic) mutual exclusion.

Explanation:
Monitors provide mutual exclusion automatically: entering any monitor procedure implicitly acquires an internal lock, and leaving it releases this lock. Because of this, no two threads can be inside the monitor's procedures at the same time, which simplifies reasoning about shared state.

SECTION 5: DINING PHILOSOPHERS
---

11. Naïve semaphore solution (possible deadlock)

a) In this solution, a deadlock can occur if all philosophers execute the `wait` on their left chopstick first at the same time.

b) The main four necessary conditions for deadlock described in operating systems textbooks
include: mutual exclusion, hold-and-wait, no preemption, and circular wait.

Explanation:
If each philosopher picks up the left chopstick first, they all successfully acquire one chopstick and then all block forever trying to acquire the right chopstick. All four conditions for deadlock hold, including circular wait (each philosopher is waiting on the next one in a cycle).

12. Asymmetric solution (avoiding circular wait)

Correct words:

- Philosopher 0 picks up the right chopstick first.
- All other philosophers pick up their left chopstick first.
- This breaks the circular wait condition for deadlock.

Explanation:
By reversing the order for one philosopher, we remove the directed cycle in the resource allocation graph. Without circular wait, deadlock cannot occur (given the other conditions remain).

13. Dining philosophers using a monitor (high level)

Fill-ins:

(1) wait()
(2) signal()

So the code lines are:

```
    self[i].wait();   // (1)
    self[i].signal(); // (2)
```

Explanation:
When a philosopher calls `pickup(i)` and cannot eat yet, they wait on their own condition variable `self[i]`. When `test(i)` discovers that philosopher `i` can eat (both neighbors are not eating and `i` is hungry), it sets `state[i] = EATING` and signals `self[i]` to wake that philosopher.

14. Starvation vs deadlock

a) A situation where some philosopher may never get to eat while others continue to eat is called starvation.

b) A situation where no philosopher can make progress (everyone is stuck waiting forever) is called deadlock.

Explanation:
Starvation is unfairness (some threads make progress, others do not). Deadlock is a state of complete standstill where none of the involved threads can proceed. The dining philosophers problem is commonly used to illustrate both possibilities and the differences between them.